Variáveis de ambiente

process.env, arquivo .env e configuração em produção

O que são variáveis de ambiente

Valores de configuração que ficam **fora do código**, no ambiente onde o programa roda.

UMA ANALOGIA

A receita é o código e é a mesma em qualquer cozinha. O que muda é o forno, os ingredientes e quem cozinha: a configuração.

`PORT`
Em qual porta o servidor escuta

`DATABASE_URL`
Onde está o banco de dados

`JWT_SECRET`
A chave que assina os tokens

O problema de escrever valores no código

```
// valores fixos dentro do arquivo

const senhaBanco = "minhaSenha123";

const chaveApi = "sk_live_9f8a7b6c";
app.listen(3000);
```

1 · VAI PARA O GIT

Quem acessa o repositório vê a senha e a chave.

2 · TROCAR DÁ TRABALHO

Para mudar um valor, é preciso editar o código e publicar de novo.

3 · UM VALOR PARA TUDO

O banco do seu computador não é o banco do servidor real.

Mesmo código, ambientes diferentes

DESENVOLVIMENTO

Seu computador

`DATABASE_URL`
`postgres://localhost/app_dev`

`FRONTEND_URL`
`http://localhost:5173`

TESTE

Testes automáticos

`DATABASE_URL`
`postgres://localhost/app_teste`

`FRONTEND_URL`
`http://localhost:5173`

PRODUÇÃO

Usuários reais

`DATABASE_URL`
`postgres://servidor-real/app`

`FRONTEND_URL`
`https://meusite.com`

> O código não muda. **Só as variáveis mudam** de um ambiente para o outro.

Lendo variáveis no Node: process.env

01-PROCESS-ENV.JS

```
// process.env é um objeto do Node

const nome = process.env.NOME;

const porta = process.env.PORT;


console.log("NOME:", nome);
console.log("PORT:", porta);
```

Um objeto com tudo

Cada variável é uma propriedade de process.env.

Não existe? undefined

Ler uma variável ausente não dá erro, só devolve undefined.

Já vem com variáveis

PATH, HOME ou USERNAME vêm do sistema operacional.

> Evite mostrar console.log(process.env) em aula ou em logs: pode expor segredos.

Exercício 1: defina no terminal

POWERSHELL

```
$env:NOME = "Ana"
$env:PORT = "4000"

node 01-process-env.js
```

CMD

```
set NOME=Ana

set PORT=4000

node 01-process-env.js
```

LINUX, MACOS, GIT BASH

```
export NOME=Ana

export PORT=4000

node 01-process-env.js
```

RESULTADO ESPERADO

```
NOME: Ana
PORT: 4000
NODE_ENV: undefined
```

Rode antes **sem definir nada** e compare: tudo sai undefined.

A variável vale só para aquele terminal. Abriu outro? Ela sumiu.

Todo valor chega como texto

O2-STRINGS.JS · COM PORT=4000 E DEBUG=FALSE

```
const porta = process.env.PORT;
console.log(porta + 1); // "40001"
console.log(Number(porta) + 1); // 4001



const debug = process.env.DEBUG;

if (debug) { /* entra! */ }

if (debug === "true") { /* certo */ }
```

Números

Converta com Number(). Sem isso, a soma vira concatenação.

Verdadeiro ou falso

O texto "false" é verdadeiro no if. Compare com "true".

Valor padrão

Use um padrão para quando faltar a variável.

```
const porta = Number(process.env.PORT) || 3000;
```

O arquivo .env

Digitar variável no terminal toda vez cansa. O **.env** é um arquivo de texto que guarda todas elas em um lugar só.

.ENV, NA RAIZ DO PROJETO

```
# configuração local, fora do Git

NOME=Ana

PORT=3000

APP_NAME=Minha API

API_KEY=troque-por-uma-chave

FRONTEND_URL=http://localhost:5173
```

NOME=valor

Uma variável por linha, sem espaço ao redor do sinal de igual.

comentário

Linhas que começam com # são ignoradas.

MAIÚSCULAS_COM_SUBLINHADO

É a convenção para os nomes.

Ao lado do package.json

Rode o node a partir dessa pasta.

Carregando o .env no Node

O Node não lê o .env sozinho. Duas formas de carregar o arquivo para dentro do process.env:

1 · NATIVO, NODE 20.6 OU SUPERIOR

```
node --env-file=.env app.js
```

Nada para instalar. Confira a versão com node -v.

2 · PACOTE DOTENV

```
npm install dotenv



import "dotenv/config";
console.log(process.env.PORT);
```

Funciona em qualquer versão e é o mais usado nos projetos.

> Se a variável já existe no terminal, **ela vence a do .env**. É assim que a produção sobrescreve a configuração local.

Exercício 2: uma API configurada pelo .env

04-SERVIDOR.JS (TRECHO)

```
import express from "express";

import cors from "cors";

import { config } from "./03-config.js";



const app = express();
app.use(cors({ origin: config.frontendUrl }));


app.get("/", (req, res) => {
  res.json({ app: config.nomeApp });
});


app.listen(config.porta);
```

1 · CRIAR O .ENV

```
copy .env.example .env
```

2 · SUBIR A API

```
npm run api
```

3 · ABRIR NO NAVEGADOR

```
http://localhost:3000
```

4 · MUDAR A PORT

Edite o .env, reinicie e abra a nova porta.

O .env nunca vai para o Git

.GITIGNORE · O GIT IGNORA O .ENV

```
node_modules

.env
```

Segredos reais ficam só na sua máquina e no servidor.

.ENV.EXAMPLE · VAI PARA O GIT

```
PORT=3000

API_KEY=troque-por-uma-chave

FRONTEND_URL=http://localhost:5173
```

Um modelo com os nomes e valores falsos, para documentar o que o projeto precisa.

QUEM CLONA O PROJETO

1. Copia o **.env.example** para **.env**. 2. Preenche com os próprios valores. 3. Roda o projeto.

Quando a variável vem undefined

O nome não bate

PORT no arquivo e process.env.PORTA no código. O nome precisa ser idêntico, inclusive maiúsculas.

Arquivo na pasta errada

O .env é procurado na pasta de onde você roda o node. Rodando de outra pasta, ele não é encontrado.

Mudou o .env e nada mudou

As variáveis são lidas só quando o programa inicia. Reinicie o servidor.

O dotenv carregou tarde

Em ESM, os imports rodam antes do resto. Use import "dotenv/config" como primeiro import.

Centralize e valide ao iniciar

03-CONFIG.JS (RESUMIDO)

```
import "dotenv/config";



const obrigatorias = ["API_KEY", "FRONTEND_URL"];

const faltando = obrigatorias.filter((n) => !process.env[n]);



if (faltando.length > 0) {
  console.error("Faltam variáveis:", faltando.join(", "));
  process.exit(1);
}



export const config = {
  porta: Number(process.env.PORT) || 3000,
  apiKey: process.env.API_KEY,
};
```

Falhe cedo

Se falta uma variável, o servidor nem sobe e diz qual é.

Um lugar só

Só este arquivo lê process.env. O resto importa config.

Converta uma vez

Number() e comparações com "true" ficam aqui.

Onde você vai usar no curso

DATABASE_URL

Endereço do banco, usado pelo Prisma

```
url = env("DATABASE_URL")
```

JWT_SECRET

Chave que assina os tokens de login

```
jwt.sign(dados, process.env.JWT_SECRET)
```

FRONTEND_URL

Origem liberada no CORS

```
cors({ origin: process.env.FRONTEND_URL })
```

PORT

Porta do servidor

```
app.listen(process.env.PORT || 3000)
```

Em produção

Variáveis de ambiente são o jeito padrão de configurar uma aplicação no servidor real.

Por que a produção depende delas

Um código, vários ambientes

O mesmo código sobe em teste e em produção. Só a configuração muda.

Segredos fora do repositório

Senhas e chaves não ficam no Git, nem com todo mundo que vê o código.

Troca sem mexer no código

Uma chave vazou? Troque o valor da variável e reinicie a aplicação.

Acesso controlado

Só quem administra a plataforma vê e edita os valores reais.

> É o princípio de config do **The Twelve-Factor App**: guarde a configuração no ambiente, não no código.

Em produção, quem preenche o process.env

DESENVOLVIMENTO · SEU COMPUTADOR

`arquivo .env` → `process.env` → Seu código

PRODUÇÃO · SERVIDOR

Painel da plataforma → `process.env` → O mesmo código

Em produção o arquivo .env normalmente **nem é enviado**. Plataformas como Render, Railway, Vercel e Heroku têm um painel para cadastrar as variáveis.

No Docker: docker run -e PORT=3000 minha-api ou --env-file.

NODE_ENV: o nome do ambiente

RODANDO: NODE_ENV=PRODUCTION NODE SERVER.JS

```
const emProducao =
  process.env.NODE_ENV === "production";



const origem = emProducao
  ? "https://meusite.com"
  : "http://localhost:5173";
```

Convenção

development, test e production.

Bibliotecas reagem a ela

O Express, em production, usa cache e mostra menos detalhes de erro.

Cada coisa no seu lugar

NODE_ENV liga e desliga comportamentos. Valores como URLs e chaves ficam em variáveis próprias.

> No projeto de exemplos, rode com NODE_ENV=production e veja o campo **ambiente** mudar na resposta da rota /.

Cuidando dos segredos

Commitou um segredo? Está vazado

Apagar o arquivo depois não limpa o histórico do Git. Gere uma chave nova e troque.

Não escreva segredos nos logs

Um console.log(process.env) em produção joga senhas dentro dos logs.

No front-end nada é secreto

O que entra no bundle, como variáveis VITE_ no Vite, fica visível no navegador. Segredo fica só no servidor.

Um valor para cada ambiente

Nunca reutilize a chave de produção no desenvolvimento nem nos exemplos da aula.

Checklist antes de publicar

- ✓ .env no .gitignore e .env.example no repositório
- ✓ Variáveis cadastradas no painel da plataforma
- ✓ Variáveis obrigatórias validadas ao iniciar
- ✓ NODE_ENV=production definido no servidor
- ✓ Chaves diferentes por ambiente, trocadas se vazarem
- ✓ Origem do CORS vinda de FRONTEND_URL, sem o asterisco

Desafio para praticar

1

Crie a variável MENSAGEM no .env e devolva o valor na rota /.

2

Faça o servidor falhar ao iniciar se a MENSAGEM não existir.

3

Crie LIMITE=2, converta para número e use para limitar uma lista.

4

Suba com PORT=4000 no terminal e prove que ele vence o .env.

Resumo

1. Variáveis de ambiente guardam a configuração **fora do código**.
2. No Node, leia com **process.env**. Todo valor chega como texto.
3. O **.env** preenche o process.env em desenvolvimento e nunca vai para o Git.
4. Centralize a leitura em um config e **valide ao iniciar**.
5. Em produção, a **plataforma** fornece as variáveis. O código continua o mesmo.